

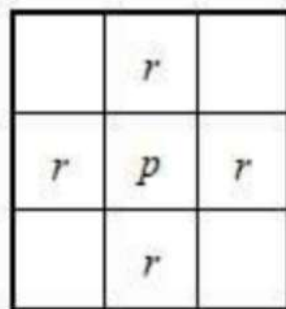


1.3.1 像素邻域

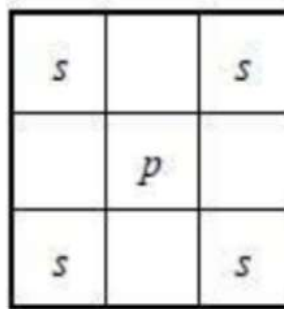
4-邻域: $N_4(p)$

对角邻域: $N_D(p)$

8-邻域: $N_8(p)$



(a)



(b)



(c)

图 1.3.1 像素的邻域



1.3.1 像素邻域

邻接：对两个像素 p 和 q 来说，如果 q 在 p 的邻域中，则称 p 和 q 满足邻接关系 **空间**

连接： p 和 q 邻接且灰度值均满足某个特定的相似准则 **空间+灰度**

连通：不（直接）邻接，但均在另一个像素的相同邻域中，且这3个像素的灰度值均满足某个特定的相似准则 **空间+灰度+空间传递**



1.3.2 像素间距离

- 距离与范数

$$D_w(p, q) = \left[|x - s|^w + |y - t|^w \right]^{1/w}$$

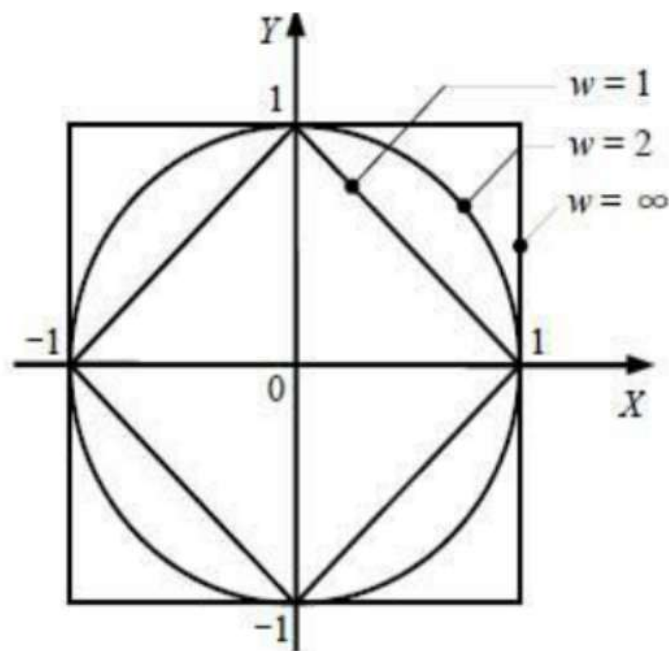


图 1.3.4 3 种范数和 3 种距离

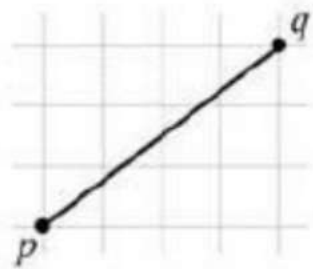


1.3.2 像素间距离

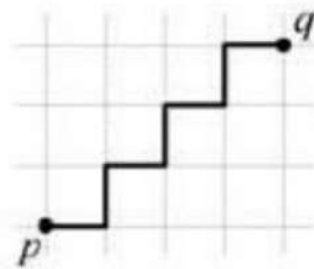
欧氏距离: $D_E(p, q) = [(x-s)^2 + (y-t)^2]^{1/2}$

城区距离: $D_4(p, q) = |x-s| + |y-t|$

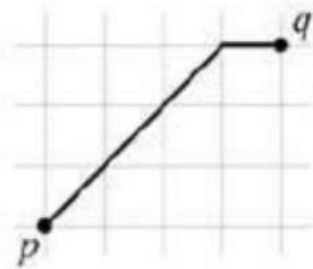
棋盘距离: $D_8(p, q) = \max(|x-s|, |y-t|)$



(a)



(b)



(c)

图 1.3.2 像素间距离的计算

欧氏距离

城区距离

棋盘距离



1.3.2 像素间距离

距离和邻域:

4-邻域: $D_4=1$ 的像素集合

p的4-邻域: $N_4(p) = \{r | D_4(p, r) = 1\}$

8-邻域: $D_8=1$ 的像素集合

p的8-邻域: $N_8(p) = \{r | D_8(p, r) = 1\}$



这里我可以理解为D4距离就是城区距离，D8距离就是棋盘距离吗？

◆ 显示思路 ▼

你的理解完全正确！

这里的 D_4 和 D_8 确实分别对应着图像处理中两种最经典的距离度量方式：



1.3.2 像素间距离

距离和邻域:

等距离轮廓: 与中心像素的某种距离等于某个值的像素集合

$\Delta_i(r), i = 4, 8$: 与中心像素的 D_i 距离小于或等于 r 的等距离轮廓

$\#[\Delta_i(r)]$: 除中心像素外 $\Delta_i(r)$ 所包含像素个数

$$\#[\Delta_4(r)] = 4 \sum_{j=1}^r j$$

$$\#[\Delta_8(r)] = 8 \sum_{j=1}^r j$$

			3				3	3	3	3	3	3	3
		3	2	3			3	2	2	2	2	2	3
3	2	1	2	3		3	2	1	1	1	2	3	
2	1	0	1	2	3	3	2	1	0	1	2	3	
3	2	1	2	3		3	2	1	1	1	2	3	
		3	2	3			3	2	2	2	2	2	3
							3	3	3	3	3	3	3



1.3.2 像素间距离

距离和邻域:

距离变换：像素与最接近的区域外像素的距离可把二值图像变换为灰度图像

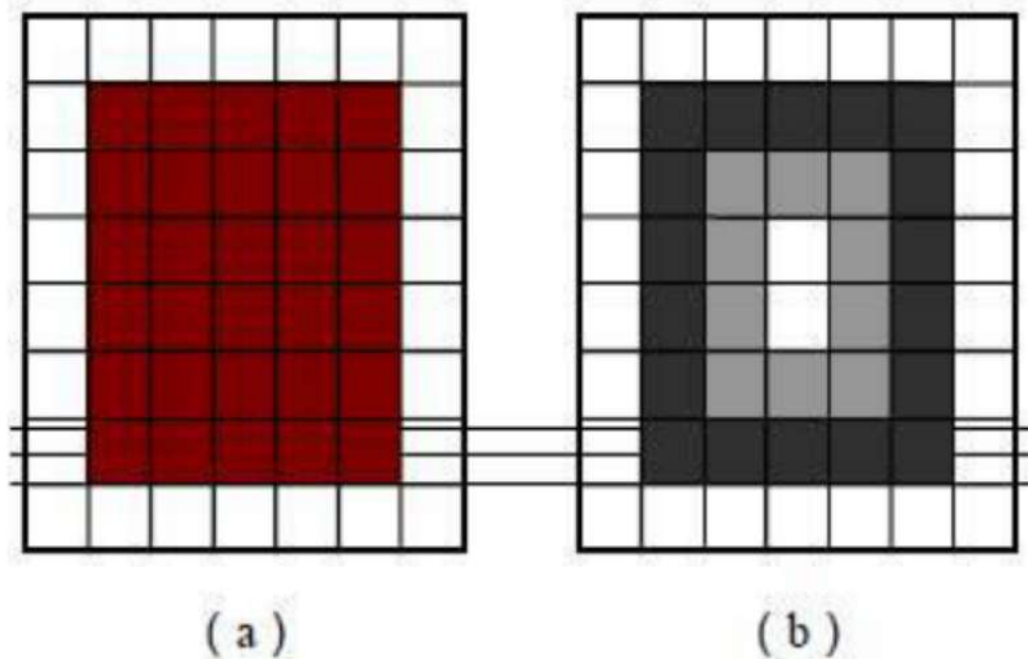
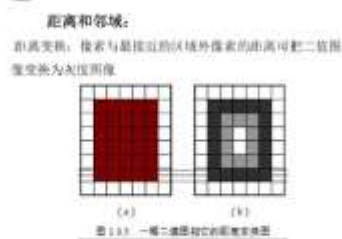


图 1.3.5 一幅二值图和它的距离变换图



这里距离变换具体是怎么去变换的？

◆ 显示思路 ▾

这张图很好地演示了**距离变换 (Distance Transform)** 的核心思想。简单来说，距离变换就是把一个二值图像（只有“内部”和“外部”之分）变成一个类似“地形图”的灰度图像，图像越亮（或灰度值越大）的地方，表示它藏得越“深”。

结合你提供的图片，具体的变换过程是这样的：

1. 明确目标与背景

- 图 (a) 是一幅二值图像：红色的方块是目标区域（前景），白色的格子是区域外（背景）。
- 距离变换的规则是：计算目标区域内的每一个像素，到离它最近的背景像素的距离。

2. 逐层计算距离（剥洋葱式变换）

我们可以像“剥洋葱”一样，从外向内看图 (b) 是如何生成的：

- **背景部分（原白色格子）**：它们本身就在区域外，距离为 0，所以在图 (b) 中保持原来的背景色不变。
- **第一层（最外围一圈）**：红色方块最外面的一圈像素，紧挨着白色的背景。它们到背景的最短距离是 1 个像素。在图 (b) 中，这一圈被赋予了距离值 1，为了区分，被渲染成了深灰色。
- **第二层（向内一圈）**：往里走一圈，这些像素到外面白色背景的最短距离变成了 2 个像素（需要跨过第一层）。在图 (b) 中，它们的距离值是 2，被渲染成了浅灰色（因为值变大了，所以颜色变亮）。
- **最中心（核心像素）**：最中间的两个像素，离所有方向的白色背景都最远，最短距离是 3 个像素。在图 (b) 中，它们的距离值最大为 3，所以被渲染成了最亮的白色。

总结

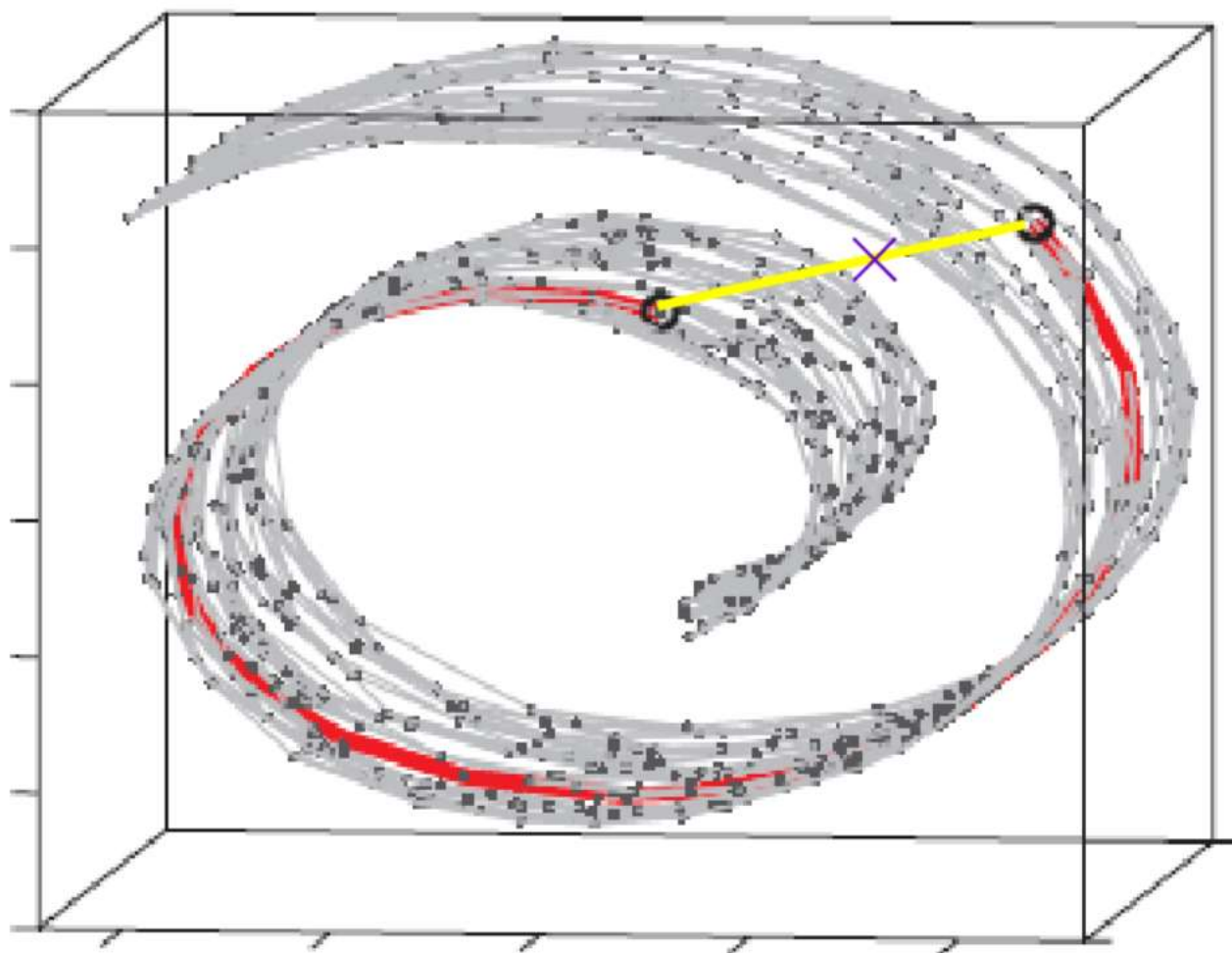
距离变换的结果图 (b) 实际上反映了原图形的骨架或中心点。颜色越亮（灰度值越高）的地方，说明这个像素在原图形状的越中心位置。图 (b) 中那种一圈一圈的矩形轮廓，也说明这里采用的距离度量方式是棋盘距离（切比雪夫距离）。



1.3.2 像素间距离

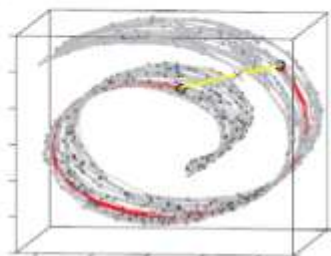


测地距离 (Geodesic Distance)



1.3.2 像素间距离

测地距离 (Geodesic Distance)



详细解释下这里的测地距离

显示思路

这张图展示的是机器学习和计算机视觉中一个非常核心且经典的概念：流形 (Manifold) 上的测地距离。

图片中用到的是极其著名的“瑞士卷” (Swiss Roll) 数据集模型。要理解测地距离，我们可以把它和我们最熟悉的直线距离做一个直接的对比。

1. 图片中的两条线分别代表什么？

- 黄色的直线 (带有一个紫色的叉 $\times$)：欧式距离 (Euclidean Distance)

这是在三维空间中两点之间“两点之间线段最短”的绝对直线距离。为什么上面打了个叉？因为在这个特定的场景下，这条线穿过了空气，它完全脱离了数据本身存在的“表面”。它没有反映出数据真实的内在结构。

- 红色的曲线：测地距离 (Geodesic Distance)

你可以把这个瑞士卷想象成一张卷起来的纸。测地距离指的是：如果你是一只蚂蚁，只能沿着这张纸的表面爬行，从一个点走到另一个点的最短路径。

2. 为什么要引入测地距离？

想象一下地球，如果你从北京去纽约，欧式距离是直接在地核里挖一条笔直的隧道穿过去。但在现实物理世界中这是不可能的，我们只能沿着地球表面 (球面) 飞过去，这个沿着表面的最短航线就是测地距离。

在你的 PPT 标题中提到这是“像素间距离”。在图像处理或高维数据分析 (流形学习) 中，数据往往不是均匀散布在整个空间里的，而是分布在某个扭曲的、低维的“子空间”或“表面”上 (就像这卷起来的二维纸)。

- 如果用欧式距离：图中那两个点在三维空间里看着离得很近，算法可能会认为它们是相似的样本。
- 如果用测地距离：算法会发现，沿着数据的实际分布走，这两个点其实离得非常远 (在纸被卷起来之前，它们可能分别在纸的两端)。

核心结论：测地距离能够揭示数据真实的内蕴几何结构 (Intrinsic Geometry)，而欧式距离往往会产生误导。这也是为什么像 Isomap 这样的经典降维算法，第一步就是要计算所有样本点之间的测地距离。



作业

- 练习题:

– 1.10

***1.10** 计算图题 1.10 中两个像素 p 和 q 之间的 D_E 距离、 D_4 距离和 D_8 距离。

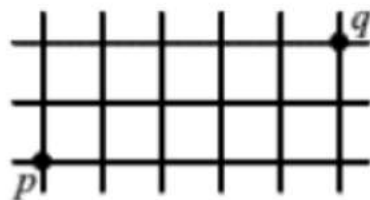


图 题 1.10

– 1.11

1.11 试分别计算 $\#[\Delta_8(4)]$ 和 $\#[\Delta_4(8)]$ 。